

SKILL WEEK 8

Task 1:

Code:

```
#include <stdio.h>
#include <string.h>

#define MAX_HISTORY 5
#define MAX_LENGTH 100

int main() {
    char history[MAX_HISTORY][MAX_LENGTH];
    char command[MAX_LENGTH];
    int count = 0;

    printf("Enter commands for history:\n");

    for (int i = 0; i < MAX_HISTORY; i++) {
        printf("Command %d: ", i + 1);
        fgets(command, MAX_LENGTH, stdin);
        command[strcspn(command, "\n")] = '\0';

        if (strlen(command) == 0)
            break;

        strcpy(history[count], command);
        count++;
    }

    printf("\nCommand History:\n");

    for (int i = 0; i < count; i++) {
        printf("%d  %s\n", i + 1, history[i]);
    }

    return 0;
}
```

OUTPUT:

venkat038@AYYAPPA:~\$

nano 8.1.c

venkat038@AYYAPPA:~\$

gcc 8.1.c -o 8.1

venkat038@AYYAPPA:~\$

./8.1

Enter commands for history:

Command 1: ls

Command 2: pwd

Command 3: date

Command 4: whoami

Command 5: clear

Command History:

1 ls

2 pwd

3 date

4 whoami

5 clear

SKILL WEEK 8

Task 2:

Code:

```
#include <stdio.h>
#include <string.h>

#define MAX_COMMANDS 5
#define MAX_LENGTH 100

int main() {
    char commands[MAX_COMMANDS][MAX_LENGTH];
    int count;

    printf("Enter number of commands: ");
    scanf("%d", &count);
    getchar();

    if (count > MAX_COMMANDS)
        count = MAX_COMMANDS;

    for (int i = 0; i < count; i++) {
        printf("Enter command %d: ", i + 1);
        fgets(commands[i], MAX_LENGTH, stdin);
        commands[i][strcspn(commands[i], "\n")] = '\0';
    }

    printf("\nPipeline Structure:\n");

    for (int i = 0; i < count; i++) {
        printf("[%s]", commands[i]);

        if (i < count - 1)
            printf(" --> ");
    }

    printf("\n");

    return 0;
}
```

OUTPUT:

```
venkat038@AYYAPPA:~$ nano 8.2.c
venkat038@AYYAPPA:~$ gcc 8.2.c -o 8.2
venkat038@AYYAPPA:~$ ./8.2
```

Enter number of commands: 3

Enter command 1: ls

Enter command 2: grep c

Enter command 3: wc -l

Pipeline Structure:

[ls] --> [grep c] --> [wc -l]

SKILL WEEK 8

Task 3:

Code:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    int pipefd[2];
    pid_t pid1, pid2;

    if (pipe(pipefd) == -1) {
        perror("pipe");
        return 1;
    }

    pid1 = fork();

    if (pid1 == 0) {
        dup2(pipefd[1], STDOUT_FILENO);

        close(pipefd[0]);
        close(pipefd[1]);

        execlp("ls", "ls", NULL);

        perror("execlp");
        return 1;
    }

    pid2 = fork();

    if (pid2 == 0) {
        dup2(pipefd[0], STDIN_FILENO);

        close(pipefd[0]);
        close(pipefd[1]);

        execlp("wc", "wc", "-l", NULL);

        perror("execlp");
        return 1;
    }

    close(pipefd[0]);
    close(pipefd[1]);

    waitpid(pid1, NULL, 0);
    waitpid(pid2, NULL, 0);

    printf("Pipeline execution completed.\n");

    return 0;
}
```

OUTPUT:

```
venkat038@AYYAPPA:~$ nano 8.3.c
venkat038@AYYAPPA:~$ gcc 8.3.c -o 8.3
venkat038@AYYAPPA:~$ ./8.3
14
Pipeline execution completed.
```

SKILL WEEK 8

Task 4:

Code:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    int pipe1[2];
    int pipe2[2];

    pipe(pipe1);
    pipe(pipe2);

    pid_t p1 = fork();

    if (p1 == 0) {
        dup2(pipe1[1], STDOUT_FILENO);

        close(pipe1[0]);
        close(pipe1[1]);
        close(pipe2[0]);
        close(pipe2[1]);

        execlp("ls", "ls", NULL);

        perror("ls");
        return 1;
    }

    pid_t p2 = fork();

    if (p2 == 0) {
        dup2(pipe1[0], STDIN_FILENO);
        dup2(pipe2[1], STDOUT_FILENO);

        close(pipe1[0]);
        close(pipe1[1]);
        close(pipe2[0]);
        close(pipe2[1]);

        execlp("grep", "grep", ".c", NULL);

        perror("grep");
        return 1;
    }

    pid_t p3 = fork();

    if (p3 == 0) {
        dup2(pipe2[0], STDIN_FILENO);

        close(pipe1[0]);
        close(pipe1[1]);
        close(pipe2[0]);
```

```

        close(pipe2[1]);

        execlp("wc", "wc", "-l", NULL);

        perror("wc");
        return 1;
    }

    close(pipe1[0]);
    close(pipe1[1]);
    close(pipe2[0]);
    close(pipe2[1]);

    waitpid(p1, NULL, 0);
    waitpid(p2, NULL, 0);
    waitpid(p3, NULL, 0);

    printf("Multiple pipeline execution completed.\n");

    return 0;
}

```

OUTPUT:

```

venkat038@AYYAPPA:~$ nano 8.4.c
venkat038@AYYAPPA:~$ gcc 8.4.c -o 8.4
venkat038@AYYAPPA:~$ ./8.4
6
Multiple pipeline execution completed.

```